

Task 2

Name: Maheen Zehra

AI-Driven Development — 30-Day **Challenge Task-2**

Part A — Theory (Short Questions):

1. Nine Pillars Understanding:

Q. Why is using AI Development Agents (like Gemini CLI) for repetitive setup tasks better for your growth as a system architect?

Answer:

AI agents (like Gemini CLI) handle repetitive setup tasks such as environment configuration and dependency installation, so we don't have to do these tasks repeatedly, and our valuable time is not wasted. This freedom allows us to focus our energy and attention on system design and architecture, during which we can decide how each component will interact and how to make the system future-ready. This approach is extremely important for our growth because it enables us to concentrate more on architecture thinking and design decisions, which are essential skills for becoming an effective system architect. The remaining repetitive setup tasks are handled by AI agents, allowing human creativity and strategic planning to be maximized.

Q. Explain how the Nine Pillars of AIDD help a developer grow into an M-Shaped Developer.

Answer:

The Nine Pillars of AIDD help a developer become an M-Shaped Developer because they teach different kinds of skills. Some pillars give vertical skills, meaning the developer gains deep expertise in a specific area, like AI integration or system design (for example, Architecture Thinking and Specification-Driven Development). Other pillars provide horizontal skills, meaning the developer learns basic or intermediate knowledge in other areas and can work better

with a team (for example, Vibe Coding, AI Agent Collaboration, and Reusability & Composability). In this way, the Nine Pillars make a developer both specialized and versatile, meaning they are an expert in one area while also being competent in several other areas, which is exactly what an M-Shaped Developer is.

2. Vibe Coding vs Specification-Driven Development:

Q. Why does Vibe Coding usually create problems after one week?

Answer:

In Vibe Coding, after a short time, the system often develops bugs and inconsistencies. It is a fast way of coding, but its drawback is that later, if new features need to be added or something needs to be updated, the code becomes difficult to understand because it was written without a clear specification or architecture. This makes making changes difficult for other developers as well. If a team is working together, everyone codes according to their own “vibe,” which makes the code messy and causes integration problems. Without proper specifications and testing, bugs and errors often appear later. These are the main reasons why Vibe Coding usually creates problems over time.

Q. How would Specification-Driven Development prevent those problems?

Answer:

In Specification-Driven Development, there are clear specifications and structure, where every feature and component is defined in a document beforehand, explaining how it should work. This makes it easier to add new features later, and other developers can easily update or change the code by referring to the specifications. When working in a team, everyone follows the same specification, so integration problems do not occur. Additionally, because of proper testing and specifications, the chances of bugs and errors are greatly reduced.

3. Architecture Thinking:

Q. How does architecture-first thinking change the role of a developer in AIDD?

Answer:

Architecture-First Thinking in AIDD changes the role of a developer so that they are no longer just writing code, but designing the entire system structure and the interactions between

components. Every decision is made not just to implement a feature, but to improve the system's long-term stability and performance. In this approach, all repetitive tasks and code generation are handled by AI agents, allowing the developer to focus entirely on design decisions and architecture thinking. This way, the developer reduces problems and makes the system future-ready, scalable, and maintainable.

Q. Explain why developers must think in layers and systems instead of raw code.

Answer:

Developers should think in systems and layers rather than raw code because it makes the code structured, which makes it easier to understand, maintain, and update. Dividing the system into layers improves scalability and makes it easier to add new features or modules. Bugs and errors are also easier to identify and fix. Additionally, team collaboration improves because each developer can work on a different layer without disturbing others. For these reasons, layered/system thinking makes the system future-ready and maintainable, whereas raw coding often leads to more errors and issues.

Part B — Practical Task (Screenshot Required)

Task:

Using any AI CLI tool, generate a 1-paragraph specification for an email validation function.

Requirements:

Must contain “@”

Must contain a valid domain (e.g., .com, .org)

Should return clear error messages

Submission:

1. Your exact CLI prompt (text or screenshot)
2. The 1-paragraph specification generated by the CLI

CLI prompt:

```
> Write a 1-paragraph specification for an email validation function. It must include an "@" symbol, a valid domain (like .com or .org), and return clear error messages for invalid input.
```

1-paragraph specification generated by the CLI:

```
> Write a 1-paragraph specification for an email validation function. It must include an "@" symbol, a valid domain (like .com or .org), and return clear error messages for invalid input.

✦ The validate_email function will accept a single string, the email address, as input. It will confirm the presence of a single "@" symbol, which must not be the first or last character. The function will also verify that the domain portion following the "@" symbol contains at least one period, and that the top-level domain (e.g., ".com", ".org") consists of at least two alphabetic characters. If the email is valid, the function will return True. For invalid input, it will return False and a concise error message specifying the reason for failure, such as "Error: Missing '@' symbol," "Error: Invalid domain name," or "Error: Email cannot be empty."
```

Part C — Multiple Choice Questions:

1. What is the main purpose of Spec-Driven Development?

A. Make coding faster

B. Clear requirements before coding begins 

C. Remove developers

D. Avoid documentation

2. What is the biggest mindset shift in AI-Driven Development?

A. Writing more code manually

B. Thinking in systems and clear instructions 

C. Memorizing more syntax

D. Working without any tools

3. Biggest failure of Vibe Coding?

A. AI stops responding

B. Architecture becomes hard to extend 

C. Code runs slow

D. Fewer comments written

4. Main advantage of using AI CLI agents (like Gemini CLI)?

A. They replace the developer completely

B. Handle repetitive tasks so dev focuses on design & problem-solving 

C. Make coding faster but less reliable

D. Make coding optional

5. What defines an M-Shaped Developer?

A. Knows little about everything

B. Deep in only one field

C. Deep skills in multiple related domains 

D. Works without AI tools